

Shared Memory

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>

int main() {
    const int size = 4096;
    const char *name = "os";
    int fd;
    char* ptr;

    fd = shm_open(name, O_RDONLY, 0666);

    ptr = (char*)mmap(0, size, PROT_READ | PROT_WRITE,
        MAP_SHARED, fd, 0);

    printf("%s", (char*)ptr);
    shm_unlink(name);
    return 0;
}

// =====PRODUCER=====
int main() {
    const int size = 4096;
    const char* name = "os";
    const char* msg1 = "hello";
    const char* msg2 = " world";

    int fd;
    char* ptr;

    fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    ftruncate(fd, size);

    ptr = (char*)mmap(0, size, PROT_READ | PROT_WRITE,
        MAP_SHARED, fd, 0);

    sprintf(ptr, "%s", msg1);
    ptr += len(msg1);

    sprintf(ptr, "%s", msg2);
    ptr += len(msg2);

    shm_unlink(name);
}
```

THREADS

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int sum = 0;
void *runner(void *parameters) {
    int i, upper = *((int*)parameters);
    if (upper > 0) {
        for(i=1; i<= upper; i++) {
            sum = sum+i;
        }
    }
    pthread_exit(0);
}

int main(int argc, char *argv[]) {
    pthread_t threadID;
    pthread_attr_t attributes;

    int num = 100;
    pthread_attr_init(&attributes);
    pthread_create(&threadID, &attributes, runner, (void*)&num);
    pthread_join(threadID, NULL);
    printf("sum= %d\n", sum);
    exit(0);
}
```

```

static volatile int counter =0;

void *myThread(void * arg) {
    printf("%s: begin\n", (char*)arg);

    int i;
    // int counter =0;
    for(i=0; i<1e7;i++) {
        counter = counter + i;
    }
    printf("%s: done, Counter: %d", (char*)arg, counter);
    return NULL;
}

int main(int argc, char* argv[]) {
    pthread_t p1,p2;

    printf("Main start, counter: %d\n", counter);
    pthread_create(&p1, NULL, myThread, "A");
    pthread_create(&p2, NULL, myThread, "B");

    pthread_join(p1, NULL);
    pthread_join(p2, NULL);

    printf("Main fone, counter: %d\n", counter);
}

```

Signals

```
#define _XOPEN_SOURCE 700

#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>
#include <sys/resoure.h>
#include <sys/wait.h>

void sigint_handler(int signum) {
    signal(SIGINT, SIG_IGN);
    fprintf(stdout, "cought signint signal (%d)\n", signum);
    exit(0);
}

int main() {
    if(signal(SIGINT, sigint_handler)==SIG_ERR) {
        printf("error setting up handler for sigint\n");
    }
    fprintf(stdout, "press ctrl+C to generate SIGINT\n");

    while(1) {
        sleep(1);
    }
    return 0;
}
```

```
void sigint_handler(int signum) {
    fprintf(stdout, "ctrl + c, sigint received\n");
    exit(signum);
}

int main() {
    struct sigaction sa;

    sa.sa_handler = sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return EXIT_FAILURE;
    }

    while (1) {
        printf("running\n");
        sleep(1);
    }

    return EXIT_SUCCESS;
}
```

```
void proc_exit() {
    int wstat;
    pid_t pid;

    while(1) {

        pid = wait3(&wstat, WNOHANG, NULL);
        if(pid==0 || pid == -1) {
            fprintf(stdout, "return value of wait3() is: %d\n", pid);
            return;
        }
        fprintf(stdout, "return value is: %d\n", wstat);
    }
}

int main9 {
    signal(SIGCHLD, proc_exit);

    switch(fork()) {
        case -1:
            perror("main: fork error");
            exit(0);
        case 0:
            printf("im runiing temporarily, \n");
            int ret = rand();
            printf("return code: %d\n", ret);
            exit(ret);
        default:
            pause();
    }
    exit(0);
}
```

Mutex & Semaphores

```
#include <pthread.h>
#include <stdio.h>

#define num_threds 5
int sharedData = 0;
pthread_mutex_t mutex;

void* thread(void *arg) {
    int thread_id = *((int*) arg);

    pthread_mutex_lock(&mutex);
    sharedData++;
    printf("Shared data: %d\n", sharedData);
    pthread_mutex_unlock(&mutex);

    pthread_exit(NULL);
}

int main() {
    pthread_t threads[num_threds];
    int threads_ags[num_threds];
    int i;

    pthread_mutex_init(&mutex, NULL);

    for (int i=0; i<num_threds; i++) {
        threads_ags[i] = i;
        pthread_create(&threads[i], NULL, thread, &threads_ags[i]);
    }
    for(int i=0; i<num_threds; i++) {
        pthread_join(threads[i], NULL);
    }
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

```

#define buffer_size 5
#define num_prod 2
#define num_con 2

int buffer[buffer_size] = {-1, -1, -1, -1, -1};
sem_t empty_slots, full_slots;
pthread_mutex_t mutex;

void* producer(void* arg) {
    int item = *((int *) arg);

    sem_wait(&empty_slots);
    pthread_mutex_lock(&mutex);

    for(int i=0;i<buffer_size;i++) {
        printf("BUFFER : %d\n",buffer[i]);
        if(buffer[i] == -1) {
            buffer[i] = item;
            break;
        }
    }

    pthread_mutex_unlock(&mutex);
    sem_post(&full_slots);

    pthread_exit(NULL);
}

void* consumer(void* arg) {
    int item = -1;
    sem_wait(&full_slots);
    pthread_mutex_lock(&mutex);

    for(int i=0;i<buffer_size;i++) {
        if(buffer[i] != -1) {
            item = buffer[i];
            buffer[i] = -1;
            break;
        }
    }

    pthread_mutex_unlock(&mutex);
    sem_post(&empty_slots);

    printf("ITEM: %d\n", item);
    pthread_exit(NULL);
}

```



```

int main() {
    pthread_t producer_threads[num_prod], consumer_threads[num_con];
    int producer_arg[num_prod] = {1, 2};

    pthread_mutex_init(&mutex, NULL);
    sem_init(&empty_slots, 0, buffer_size);
    sem_init(&full_slots, 0, 0);

    for(int i=0;i<num_prod;i++) {
        pthread_create(&producer_threads[i], NULL, producer, &producer_arg[i]);
    }
    for(int i=0;i<num_con;i++) {
        pthread_create(&consumer_threads[i], NULL, consumer, NULL);
    }

    for(int i=0;i<num_prod;i++) {
        pthread_join(producer_threads[i], NULL);
    }
    for(int i=0;i<num_con;i++) {
        pthread_join(consumer_threads[i], NULL);
    }

    pthread_mutex_destroy(&mutex);
    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);

    return 0;
}

```

Bounded buffer

```

#define BUFFER_SIZE 5
sem_t mutex, empty, full;
int buffer[BUFFER_SIZE];
int in = 0, out = 0;

void *producer(void *arg) {
    int item;
    while (1) {
        item = rand() % 100; // Generate a random item to produce
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&full);
        sleep(rand() % 3); // Simulate some processing time
    }
}

void *consumer(void *arg) {
    int item;
    while (1) {
        sem_wait(&full);
        sem_wait(&mutex);
        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&empty);
        sleep(rand() % 3); // Simulate some processing time
    }
}

int main() {
    pthread_t producer_thread, consumer_thread;

    sem_init(&mutex, 0, 1);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);
}

```

Reader-Writer

```
#define NUM_READERS 5
#define NUM_WRITERS 2
#define STRING_LENGTH 60

pthread_t readers[NUM_READERS], writers[NUM_WRITERS];
sem_t mutex, rw_mutex;
int readers_count = 0;
FILE *file;

char generateRandomChar() {
    return (char)('a' + rand() % 26); // Generating a random lowercase character
}

void *reader(void *arg) {
    while (1) {
        sem_wait(&mutex);
        readers_count++;

        if (readers_count == 1) {
            sem_wait(&rw_mutex);
        }

        sem_post(&mutex);
        // Reading from file
        fseek(file, 0, SEEK_SET);
        char buffer[256];
        while (fgets(buffer, sizeof(buffer), file) != NULL) {
            fprintf(stdout, "Reader %ld: %s", (long)arg, buffer);
        }
        sem_wait(&mutex);
        readers_count--;
        if (readers_count == 0) {
            sem_post(&rw_mutex);
        }
        sem_post(&mutex);

        // Perform other tasks
        usleep(1000);
    }
}
```

```

void *writer(void *arg) {
    while (1) {
        sem_wait(&rw_mutex);
        // generate random string to be written to file
        srand(time(NULL)); // Seed for random number generator
        char randomString[STRING_LENGTH + 1]; // +1 for null terminator
        for (int i = 0; i < STRING_LENGTH; i++) {
            randomString[i] = generateRandomChar();
        }
        randomString[STRING_LENGTH] = '\0'; // Null terminate the string
        // Writing to file
        fseek(file, 0, SEEK_END);
        fprintf(file, "%s\n", randomString); // writer to file on drive
        fprintf(stdout, "Writer %ld: %s\n", (long)arg, randomString); // display
        fflush(file);
        sem_post(&rw_mutex);
        // Perform other tasks
        usleep(1000);
    }
}

int main() {
    file = fopen("shared_file.txt", "a+");
    if (file == NULL) {
        perror("Error opening file");
        exit(EXIT_FAILURE);
    }

    sem_init(&mutex, 0, 1);
    sem_init(&rw_mutex, 0, 1);
    int i;

    for (i = 0; i < NUM_WRITERS; i++) pthread_create(&writers[i], NULL, writer, (void *) (long)i);
    for (i = 0; i < NUM_READERS; i++) pthread_create(&readers[i], NULL, reader, (void *) (long)i);

    for (i = 0; i < NUM_READERS; i++) pthread_join(readers[i], NULL);
    for (i = 0; i < NUM_WRITERS; i++) pthread_join(writers[i], NULL);

    fprintf(stdout, "reader pthread join completed\n");
    sem_destroy(&mutex);
    sem_destroy(&rw_mutex);
    fclose(file);
    return 0;
}

```